

SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-5-WS2025-INFC-EN

laC Tool

Ausgeführt von: David Veigel

Kevin Forter

Personenkennzeichen: 2310257030,
2500257003

BegutachterIn:

Wien, den 2. Januar 2026

Inhaltsverzeichnis

1 Theoretical introduction	1
1.1 Progress Chef	1
1.2 Ansatz und Architektur	1
1.3 Vergleich mit Terraform und Ansible	2
1.4 Lizenzmodell	2
1.5 Server Virtualization, Support and Performance	2
1.6 Integration mit anderen Werkzeugen	2
2 Praktische Umsetzung mit Chef Infra Client	3
2.1 Voraussetzungen und Setup	3
2.2 Nachbildung von Terraform Workshop 1	3
2.3 Nachbildung von Terraform Workshop 2	5
2.4 Nachbildung von Terraform Workshop 3	5
2.5 Nachbildung der Ansible Workshops	6
2.6 Automatisierung und Wiederholbarkeit	7
2.7 Zusammenfassung der praktischen Ergebnisse	7

1 Theoretical introduction

1.1 Progress Chef

Progress Chef ist eine Plattform zur Automatisierung und Verwaltung von IT-Infrastrukturen. Sie ermöglicht es Administratoren und DevOps-Teams, den gewünschten Zustand von Systemen zu definieren und diesen automatisiert durchzusetzen.

Typische Einsatzgebiete von Chef sind:

- Automatisierte Serverkonfiguration
- Applikationsbereitstellung
- Verwaltung von Systemzuständen
- Vermeidung von Konfigurationsabweichungen

1.2 Ansatz und Architektur

Chef basiert auf dem Infrastructure-as-Code-Prinzip und verwendet ein deklaratives Modell. Konfigurationen werden mithilfe einer Ruby-basierten Domain Specific Language (DSL) beschrieben.

Chef kann in zwei Betriebsmodi eingesetzt werden:

- **Client-Server-Modus:** Die verwalteten Systeme beziehen ihre Konfigurationen von einem zentralen Chef Server.
- **Lokaler Modus (Chef Solo):** Die Konfigurationen werden lokal ohne zentrale Instanz ausgeführt.

Zentrale Bestandteile von Chef sind:

- **Cookbooks:** Sammlungen von Konfigurationslogik
- **Rezepte:** Konkrete Konfigurationsanweisungen
- **Ressourcen:** Abstraktionen von Systemkomponenten
- **Chef Client:** Agent zur Anwendung der Konfigurationen

1.3 Vergleich mit Terraform und Ansible

Chef unterscheidet sich deutlich von Terraform und Ansible hinsichtlich seines Einsatzzwecks.

- **Terraform** wird primär zur Bereitstellung von Infrastrukturressourcen verwendet.
- **Ansible** eignet sich besonders für Orchestrierung und einfache Konfigurationsaufgaben.
- **Chef** ist auf langfristige Konfigurationsverwaltung und Skalierbarkeit ausgelegt.

In der Praxis wird Chef häufig in Kombination mit Terraform eingesetzt, wobei Terraform die Infrastruktur erstellt und Chef die anschließende Konfiguration übernimmt.

1.4 Lizenzmodell

Der Chef Infra Client ist als Open-Source-Software unter der Apache-2.0-Lizenz verfügbar. Kommerzielle Komponenten wie Chef Infra Server oder Chef Automate werden von Progress Software im Rahmen eines Lizenzmodells angeboten. Für kleinere Umgebungen existiert ein kostenfreies Nutzungskontingent.

1.5 Server Virtualization, Support and Performance

- Speziell für **Server-Virtualisierung** konzipiert.
- Unterstützt sowohl **Windows**- als auch **Linux**-Gastsysteme.
- Ressourcenlimits abhängig von der Hardware:
 - Bis zu **288 CPUs** pro Host
 - Bis zu **5 TB RAM** pro Host
 - Pro VM: typischerweise bis zu **32 vCPUs** und **128 GB+ RAM** (konfigurierbar)
- Clusterverwaltung über **Xen Orchestra** oder **XCP-ng Center**.

1.6 Integration mit anderen Werkzeugen

Chef lässt sich in zahlreiche bestehende Systeme integrieren, darunter:

- Cloud-Plattformen wie AWS, Microsoft Azure und Google Cloud
- Container-Technologien wie Docker und Kubernetes
- CI/CD-Werkzeuge wie Jenkins oder GitHub Actions
- Infrastruktur-Tools wie Terraform

2 Praktische Umsetzung mit Chef Infra Client

Ziel des praktischen Teils war es, die in den Terraform- und Ansible-Workshops definierten Ziele mithilfe des Chef Infra Clients nachzubilden. Da Chef kein Infrastruktur-Provisionierungswerkzeug ist, wurde die zugrundeliegende Infrastruktur vorab über die AWS Academy bereitgestellt. Chef übernahm anschließend die Konfigurationsverwaltung, Server-Provisionierung und Automatisierung.

2.1 Voraussetzungen und Setup

Für die praktische Umsetzung wurden folgende Voraussetzungen geschaffen:

- AWS Academy Account mit Zugriff auf EC2
- Ubuntu Linux EC2-Instanzen (t2.micro)
- Öffentliche Sicherheitsgruppe mit Zugriff auf Port 22 (SSH) und Port 80 (HTTP)
- Installation des Chef Infra Clients auf allen Zielsystemen



Abbildung 1: AWS EC2 Dashboard mit laufenden Instanzen

2.2 Nachbildung von Terraform Workshop 1

Im ersten Terraform-Workshop wurde eine EC2-Instanz mit Apache-Webserver bereitgestellt. Diese Funktionalität wurde mithilfe des Chef Infra Clients nachgebildet, indem eine bestehende EC2-Instanz konfiguriert wurde.

Hierzu wurde ein eigenes Cookbook erstellt, welches die Installation und Konfiguration des Apache-Webrowsers automatisiert.

```
package 'apache2'

service 'apache2' do
  action [:enable, :start]
end

file '/var/www/html/index.html' do
  content '<h1>Hello_World</h1>'
  mode '0644'
end
```

Listing 2.1: Chef-Rezept zur Installation und Konfiguration von Apache

Die Ausführung des Chef Clients erfolgte im lokalen Modus:

```
chef-client --local-mode --runlist 'recipe[apache]'
```



Abbildung 2: Ausführung des Chef Infra Clients zur Apache-Installation

Anschließend wurde die Erreichbarkeit des Webrowsers über den Browser überprüft.

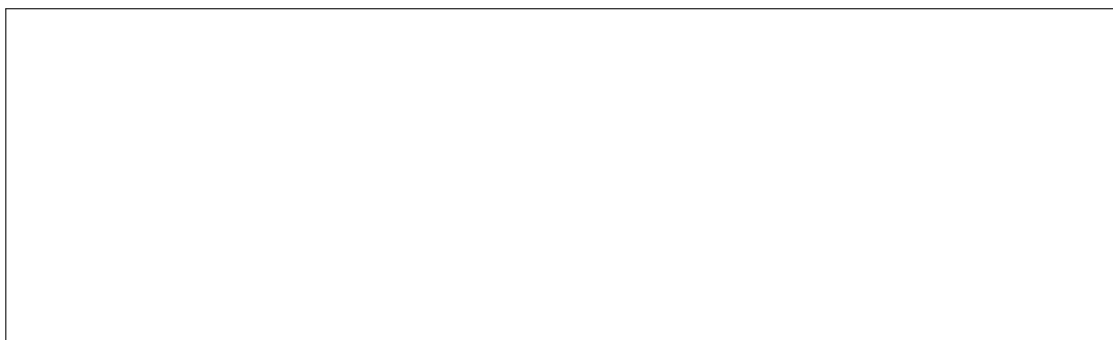


Abbildung 3: Erreichbarkeit des Apache-Webrowsers über die öffentliche IP-Adresse

2.3 Nachbildung von Terraform Workshop 2

Der zweite Terraform-Workshop erweiterte die Infrastruktur um benutzerdefinierte Netzwerkkomponenten wie VPCs, Subnetze und Elastic IPs. Diese Komponenten liegen außerhalb des Verantwortungsbereichs von Chef und wurden daher vorab über die AWS Academy konfiguriert.

Chef wurde anschließend eingesetzt, um die Konfiguration der bereitgestellten Instanzen automatisiert und reproduzierbar umzusetzen. Das gleiche Cookbook wie im ersten Workshop konnte unverändert wiederverwendet werden, was die Wiederverwendbarkeit und Idempotenz von Chef unterstreicht.



Abbildung 4: Vorkonfigurierte Netzwerkstruktur in AWS

2.4 Nachbildung von Terraform Workshop 3

Im dritten Terraform-Workshop wurde die Skalierung auf mehrere Instanzen sowie der Einsatz eines Load Balancers thematisiert. Zur Nachbildung dieser Funktionalität wurden mehrere EC2-Instanzen erstellt und identisch mit Chef konfiguriert.

Eine zusätzliche Instanz fungierte als Reverse Proxy. Chef installierte und konfigurierte dort einen NGINX-Webserver, welcher die Anfragen auf die Backend-Instanzen verteilte.

```
package 'nginx'

file '/etc/nginx/sites-enabled/default' do
  content <<~EOF
  upstream backend {
    server 10.0.0.10;
    server 10.0.0.11;
  }

  server {
    listen 80;
    location / {
      proxy_pass http://backend;
    }
  }
endfile
```

```

    }
    EOF
    notifies :restart, 'service[nginx]'
end

service 'nginx' do
  action [:enable, :start]
end

```

Listing 2.2: Chef-Rezept zur Konfiguration eines NGINX Load Balancers



Abbildung 5: NGINX Load Balancer Konfiguration auf der Proxy-Instanz

Die öffentliche IP-Adresse des Load Balancers wurde anschließend verwendet, um den Zugriff auf die dahinterliegenden Webserver zu testen.

2.5 Nachbildung der Ansible Workshops

Die in den Ansible-Workshops vermittelten Konzepte zur Konfigurationsverwaltung und Mehrknoten-Administration konnten nahezu vollständig mit Chef nachgebildet werden.

Mehrere EC2-Instanzen wurden mit dem Chef Infra Client ausgestattet und mit identischen Cookbooks konfiguriert. Chef ersetzte hierbei die Ansible-Playbooks durch deklarative Rezepte und Cookbooks.

Zusätzlich wurden separate Cookbooks für Apache und MySQL erstellt, um eine einfache LAMP-ähnliche Umgebung bereitzustellen.

```

package 'mysql-server'

service 'mysql' do
  action [:enable, :start]
end

```

Listing 2.3: Chef-Rezept zur Installation von MySQL



Abbildung 6: Automatisierte Installation von MySQL mit Chef

2.6 Automatisierung und Wiederholbarkeit

Die Cookbooks wurden in einem Git-Repository versioniert. Über eine manuell gestartete CI/CD-Pipeline konnten Chef-Runs automatisiert ausgelöst werden. Dieses Vorgehen ersetzt funktional die in den Terraform-Workshops verwendeten manuellen Workflows.



Abbildung 7: Manuell gestarteter GitHub-Workflow zur Ausführung von Chef

2.7 Zusammenfassung der praktischen Ergebnisse

Die praktische Umsetzung zeigte, dass Chef Infra Client in der Lage ist, die Konfigurations- und Automatisierungsziele der Terraform- und Ansible-Workshops zu erfüllen. Während die Erstellung von Infrastruktur weiterhin außerhalb des Funktionsumfangs liegt, ermöglicht Chef eine konsistente, skalierbare und wiederholbare Systemkonfiguration.